

Dual Petri Epistricted Liveness Protocol

Paul L. Borrill

2022-Jun-26

Dual-SAW-Spekkens.pdf

Maintaining liveness and synchronizing processes in network channels is a perennial challenge when packets can be dropped, reordered, duplicated or delayed. Conventional switched networks require protocol stacks and applications to use timeouts and retries to maintain liveness. This makes exactly-once semantics impossible and precipitates retry storms which lead to unbounded tail latency and transaction failure.

A superior solution is to employ individual direct channels with the Stop and Wait (SAW) protocol to maintain flow control, [Colored Petri Nets](#)¹ (CPTs) to process different actions (for different orientations), and the Spekkens' Toy Model (STM) to manage [epistricted](#) consistency^{2,3,4} for distributed metadata.

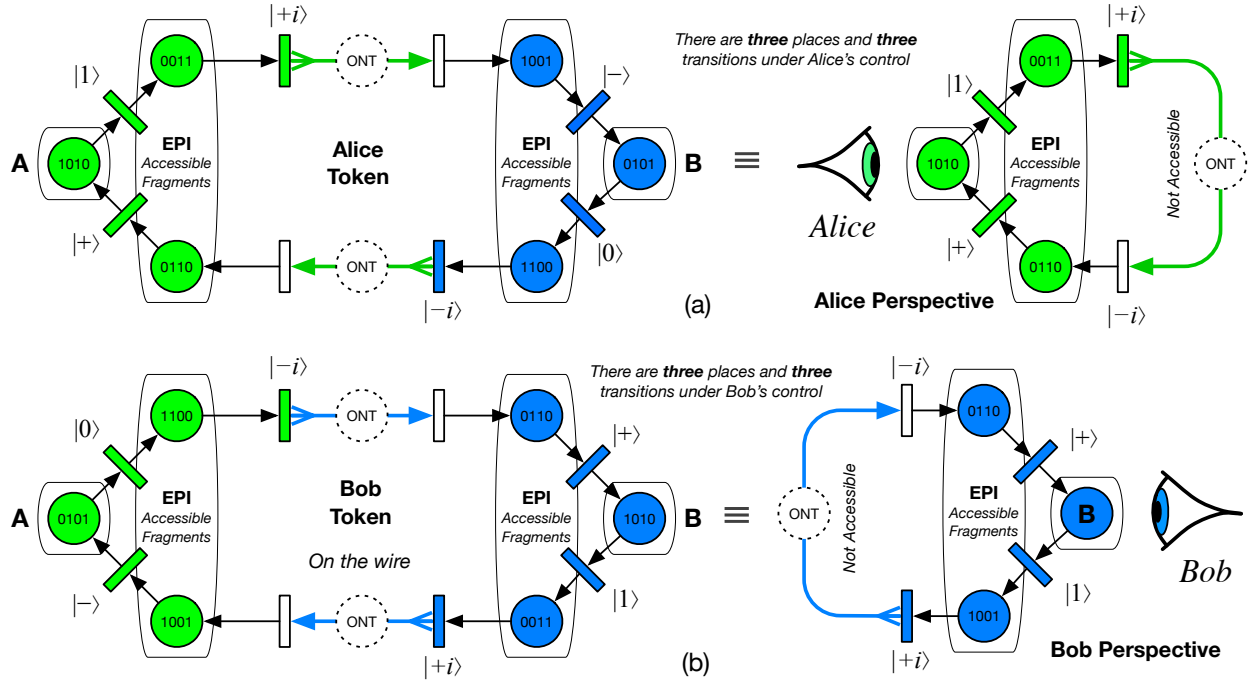


Figure 1: Single Channel. Dual SAW Petri-Spekkens (DSPS) Protocol

A single circulating (black) Petri token with an *even* number of Petri places and actions will create a stable sequence of actions with the same unchanging state at each transition and place in this Petri Net. Using *two* tokens, one *owned* by Alice and one *owned* by Bob (Fig. 1) will exhibit alternating ‘dynamic’ liveness at places A and B. Is entanglement when *two* photons/packets/Petri tokens are in flight *simultaneously*?

Using colored tokens, *serializability* for distributed algorithms is resolved for Alice using her token, for Bob using his token, and either waiting for (two phase locking) both tokens. Applications specify ordering constraints, and the interface can ‘hold’ tokens to reliably compose *linearizable* ‘application primitives’. The model generalizes to any number of different color tokens or any number of stages in a chain, tree or graph.

Alice controls serializability *from her perspective*, by using her Petri token. The same for Bob, using his token. When the ordering of events needs to be resolved for an algorithm, cell-agents wait for all the tokens necessary to resolve the uncertainty. Do we want Alice’s to occur before Bob’s, or Bob’s before Alice’s, or does Carol choose the order for them? An easy constraint for applications to specify. This provides a deterministic primitive which the software layers above can request, related to application consistency.

In effect, (inside a network enclave/FPGA Substructure) we ‘collapse’ the network to a consistent state by allowing only those orders that are specified by the application as valid. We call this ‘discrete’ superposition.

¹Samson Abramsky: [Petri Nets, Discrete Physics, and Distributed Quantum Computation](#).

²Bob Coecke, Bill Edwards: [Toy Quantum Categories](#).

³Robert Spekkens: [Quasi-quantization: classical statistical theories with an epistemic restriction](#).

⁴John H. Selby, David Schmid, Elie Wolfe, Ana Belén Sainz, Ravi Kunjwal, Robert W. Spekkens: [Accessible fragments of generalized probabilistic theories, cone equivalence, and applications to witnessing nonclassicality](#).

Host Processor Superstructure and FPGA Substructure

In the spirit of Quantum Alice and Silent Bob⁵ we meticulously separate the information (and events) occurring in the FPGA substructure from being seen by (or interacting with) the host superstructure on the other side of the PCIe bus – except under very strict circumstances. A typical assumption is that Qubit-based Quantum Key Recycling (QKR) would require a quantum computer or classical ciphertext. We see a third alternative: by using Spekkens’ toy qubits, as ‘metadata’ in the interface between the host processor in the superstructure, and the FPGA logic in the substructure, we can achieve many security benefits by exploiting the no-cloning and teleportation properties. Physically Unclonable Functions⁶ (PUFs) now available in the FPGA, enable far better ‘confinement’ of security-sensitive properties in the substructure.

Serializability

Figure 2 Shows a Petri Net for two links and three cells: (C_1, C_2, C_3). EPI information is represented by the last place before the packet goes out on the wire, and the first place where a packet comes in on the wire. Each Link is symmetric, we refer to alice (a), and bob (b), lower-case letters. Host Cells (C) (Carol or Charlie are at the next level up in the architecture. Upper case characters refer to cell/node/host names. In addition to being referred to as Carol or Charlie within the cell, they also have a cell ID (provided by a Physically Unclonable Function within the FPGA).

Each Link Has:

One Bipartite Pair connected by one *logical* link (made up of two transmit-receive pairs).

Two connected ports One transmit-receive pair for each direction. Alice and Bob symmetry is broken within the link by assuming those roles are “self-and other”.

Two Circulating Frames One owned by alice, one owned by bob. Described in [Requirements.pdf](#)

We ‘could’ violate the Petri net rule of alternating places and transitions to accommodate the Spekkens Toy model; where there may not be a place (ONT) that can be observed. The two transitions are theoretically equivalent to the transmission of a packet, and the reception of a packet, This is because packets may be sent but not received, so there is a natural failure mode at that point.

Instead we show the ‘ONT’ place between those two transitions with a dotted outline to indicate that this information is not accessible. EPI information (last storage before going out on the wire, first storage coming in from the wire) is accessible for error recovery and tree-healing.

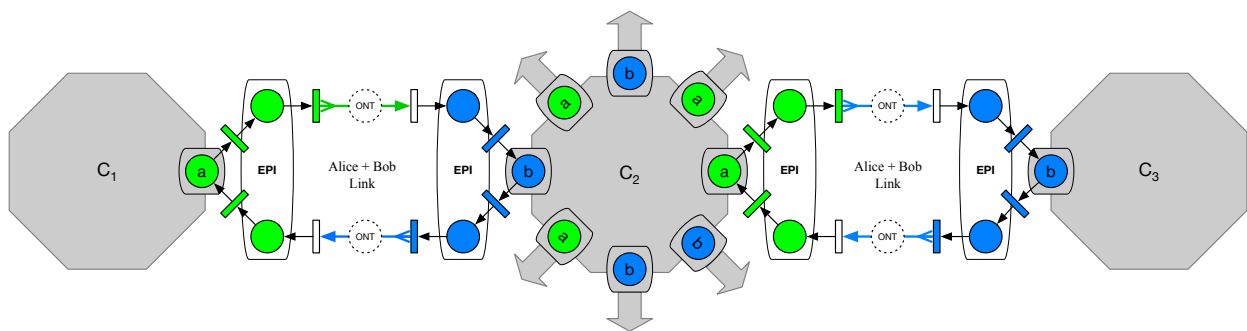


Figure 2: A Compositional Liveness Chain: C_1, C_2, C_3 with two Channels: $a \rightleftharpoons b$ (between C_1 and C_2) and another $a \rightleftharpoons b$ (between C_2 and C_3). Each Link has independent circulating tokens for their internal alice and bob. Any Charlie (connecting two or more ports) can synchronize information from one link to the other.

- Composing these channels into a chain (univalent C_1 and C_3 , bivalent C_2).
- For each channel, there are two atoms (one owned by alice, one owned by bob).

⁵Quantum Alice and Silent Bob: Qubit-based Quantum Key Recycling with almost no classical communication.

⁶Physical Unclonable Functions (PUFs) are devices that exploit inherent randomness in the manufacturing process to give a physical entity a unique ‘fingerprint’ or trust anchor. See: [Physical unclonable functions](#) and [Hybrid Boolean Networks](#).

- Sharing the tokens across all links. E.g., C_2 (in the middle), ‘forwards’ ‘Cell tokens’ (atoms) from the link on the left, to (atoms) the link on the right so they can be observed on both sides. ‘Cell Tokens’ will require a Cell ID to be attached to the packet so it is otherwise distinguishable to separate cells.
- Links share atoms (alice and bob atomic units). Cells share ‘Cell tokens’.

Each Cell Has:

Eight Logical Ports Each pair a and b acts independently until Charlie (C) coordinates transfers between ports.

Coordinator Charlie (Fast forwarder). Preselects (via forwarding table), what slices are automatically sent from some incoming port to some outgoing port. See [Slice-Engine.pdf](#) for details.

Captain Charlie (Reordering engine). See [Slice-Engine.pdf](#) for details.

Six Distinguishable Event Orderings – for Alice or Bob symmetric orientation

Fig. 3 shows six alternative ways the Petri tokens can arrive and depart on a single Channel (time - bit progression - goes down the page when viewed from the SmartNIC SERDES). When measured (M) in the input slice (1/8th of an Ethernet Frame) they depict (0) no token, (+1) token arrives, and (−1) token departs. When both tokens are present, the sum of the two states is +2. The Spekkens’ toy qubits⁷ may be used to distinguish six alternatives.

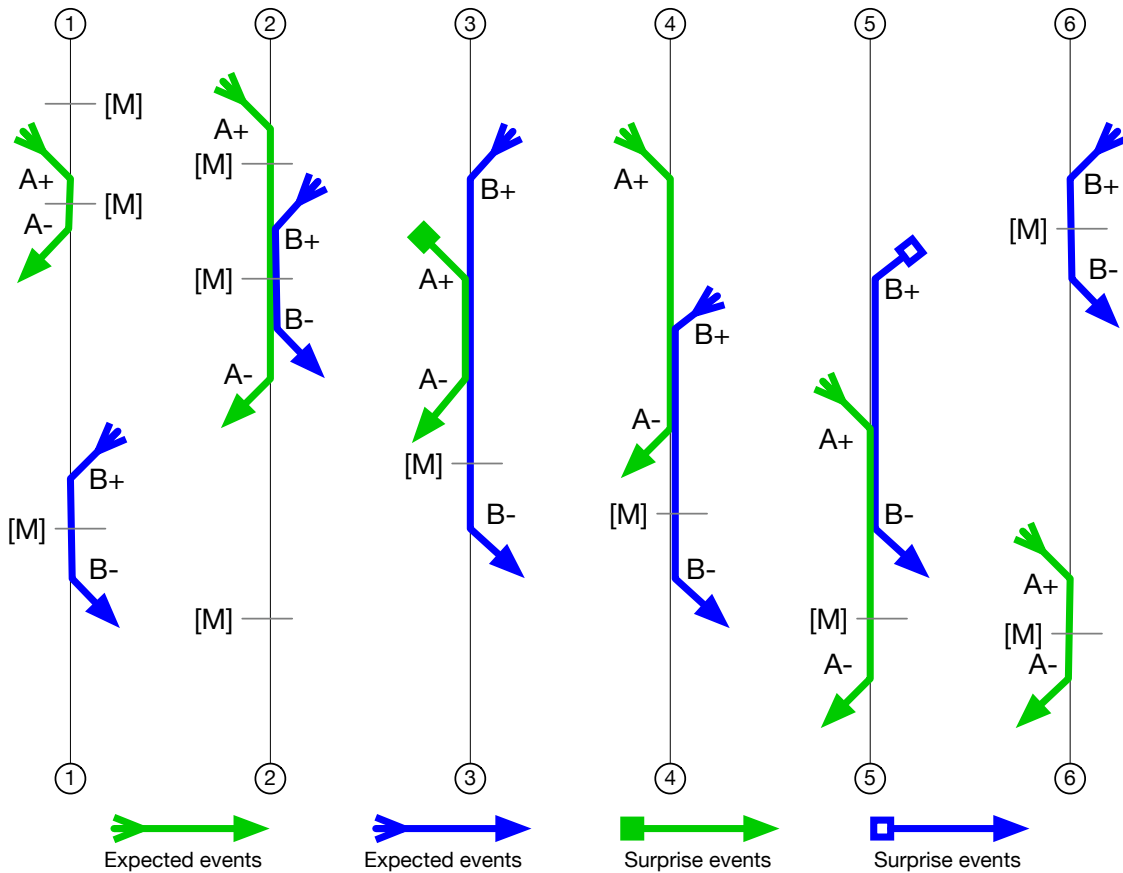


Figure 3: Petri Net Token Event Orderings (Spekkens Alternatives)

⁷Disilvestro & Markhaam: [Quantum protocols with Spekkens Toy Model](#).

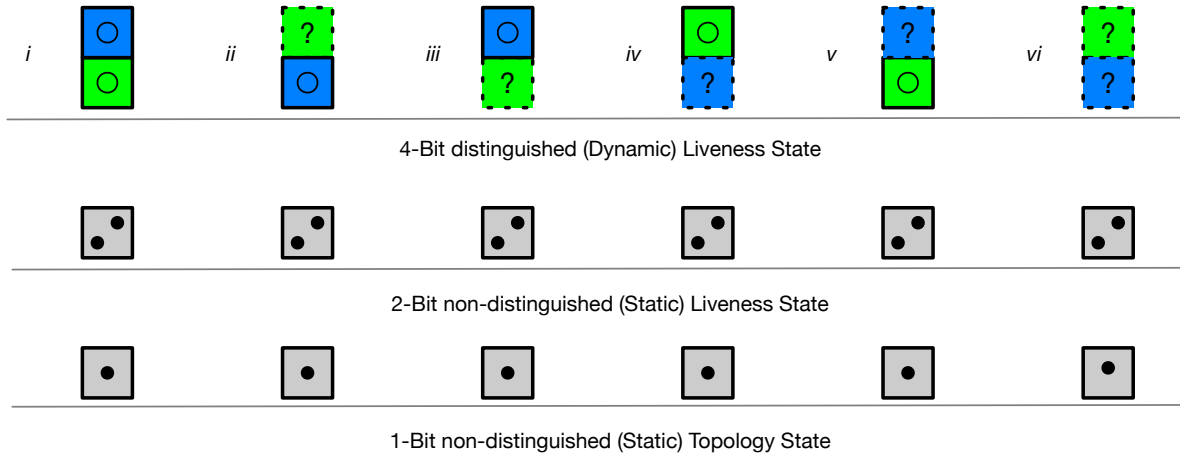


Figure 4: Discrete Superposition of two tokens on one Channel.

XOR and Spekkens Bits

Critical to Spekkens' argument is that it's the epistemic states, and not the ontic ones, that resemble [qubits](#). Here, we tie two entangled *toy* qubits for each interaction pair (what we call MORTS – Morphism pORTS). The key challenge is being precise about what it represents (and what it doesn't).

By creating a deliberate epistemic restriction, Spekkens' toy model has been proven to exhibit several what were previously considered 'quantum-only' behaviors. Critically (for our purposes here) this includes no-cloning and teleportation. By recognizing the arrival of events (Petri tokens), we can distinguish (inside the substructure implementation) the event orderings, but only by setting up the system to ask the right yes/no question. This sequence begins with the 'liveness architecture' described in a separate document.

Spekkens Epistemic Restrictions

We specify Spekkens⁸ epistemic/ontological boundary in a Petri net by two consecutive Petri transitions (the action mechanism which fires the Petri net to transfer tokens) connected in series without an intervening place, because this information cannot be observed (therefore it cannot exist and be available to the programmer). However, the epistemic information at A, B, and C can.

This can be generalized to multiport FPGA SmartNICs, by imagining that each port on the network (e.g. 8 transceivers operating at 116Gb/s on the most recent Intel M-Series FPGA's), is (one of eight) individual channels, each part of an $(N - 1)$ Chain of channels along the channel axis. And there are M such channels that can operate in parallel (completely concurrently in ordering the events on this multiplicity of ports).

The total number of Spekkens bits required for a cluster of 9 connected cells (the self-cell and its 8 connected neighbors), is thus $4 \times N \times M$. 4-bits for each entangled token, twice that for two tokens.

Accessible (Observable) State (EPI)

The 'EPI fragments' is the combination of the last Petri state before the token goes out on the wire, and the first Petri state that comes off the wire in the receiver. This is at least compatible in principle with Spekkens' view of the world, and although descriptions may vary from person to person based on their preferred interpretation (of quantum foundations), and philosophical leanings, just like with our understanding of real quantum mechanics, the math is consistent, and describes a radically simple state of affairs that can easily be implemented in a packet encoding and processed by an FPGA in a SmartNIC.

The key to maintaining the key properties of

⁸The [Spekkens' knowledge balance principle](#) states that for a system which requires $2N$ bits of information to completely specify, only N bits of information may be known about the system at any time. The state of the system is called the ontic state, while the state of our (limited) knowledge about the system is called the epistemic state.

Inaccessible (Unobservable) State (ONT)

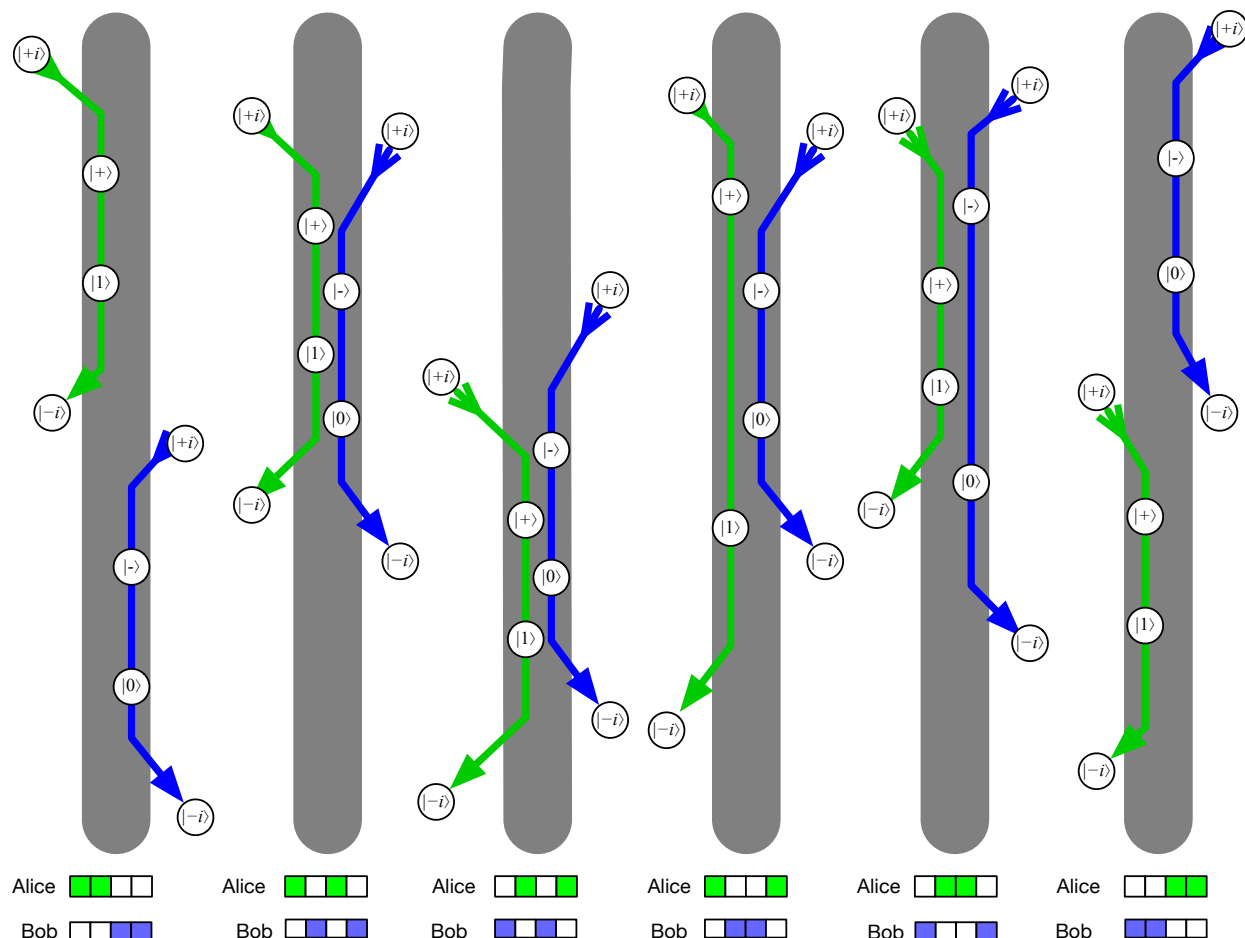
The Petri transitions are *events*, as well as *actions* in the language of Petri Nets. One represents sending a packet, the other represents receiving the same packet. They are ‘separable’ because it is possible, due to some failure, that the packet was sent, but not received. This happens frequently in conventional switched networks, because dropping packets is accepted behavior of switches and routers. But even in a directly connected network, it is still possible for the Channel to be disconnected, or the packets to be corrupted – even though this may be vastly less probable than the behavior of switches in dropping, reordering, duplicating and delaying packets.

Race Conditions $\equiv$ Uncertainty

There is a race condition in Figure 2 (Alice Bob Charlie). Alice/Bob tokens and Bob/Charlie tokens arriving at Bob represent a fundamental uncertainty (a race condition). While Figure 4 represents the two party superposition, Figure 2 is a three party operation, and has a greater number of states.

We make a distinction between the low-level Petri Net (black) tokens, which are not conserved, so we use black tokens purely to maintain liveness, and not for the EQ/CQ mechanism.

Bob and Alice Only: Six Spekkens Ordering Alternatives



Petri-Spekkens Token Orderings in the FPGA SerDes and Disjoint Pairs

Figure 5: Six Petri Spekkens sequencing alternatives. What Order Would You Like To Ask Your Questions?

This is the view (from either Alice or Bob) of the token arrival and departures. Observations:

1. From Alice's observation point, she knows that her serializability token is owned by her, and 'nothing new' can happen without her initiating it. Alice has no such guarantees with Bob's token, because Bob is the 'decider' of what happens with the next version of his token.
2. From Bob's observation point, he knows that his serializability token is owned by him, and 'nothing new' can happen without him initiating it. Bob has no such guarantees with Alice's token, because Alice is the 'decider' of what happens with the next version of her token.

Note that there are two operators (questions that can be asked of the epistricted knowledge system), shown here as a bit flip operation or a phase flip operation. While the details of this will matter in a single photon quantum experiment, this principle can still be employed by the 'event structures' of the FPGA by moving into the computer science / encoding space, and employing different 'types' of tokens.

Alice can select (the equivalent of) a bit flip operation, or a phase flip operation. These operations are non-commutative in real quantum measurements. They can also represent non-commutative operations in the modification of data structures in our toy system. The symmetry would be destroyed, and an *unfair imposition* decided by one side or the other if we allowed, in this bipartite toy entanglement, for Bob, or Alice, to make such a decision.

This means, therefore, that a third party is essential if we are to make a useful measurement (one that mimics quantum behavior classically). In the following pages, we distinguish between Carol, the referee (or adjudicator) to 'arbitrate' which of these events occurred first, and which occurred second.

We also add a new concept: made possible by the colored Petri tokens owned by Alice and Bob, who we call 'Charlie'. Charlie can do more than be a referee. He can (if needed) swap the order of the questions being asked, making an otherwise non-commutative operation commutative according to the requirements specified by application fragments.

Bob, Alice: A View from Captain Charlie

This is the view (from Charlie) of the Alice and Bob tokens arriving and departing at her cell (on different links, OR on the same link, connected (composed) from multiple links. Observations:

1. If this 3rd entity was a 'Carol?' who could only observe, then we would have a non-participating referee – an independent arbiter for the arrival and departure of tokens. She could, for example, act as a referee for who came first, whether they overlapped or not. The tripartite model is much more interesting.
2. If this 3rd entity has computational authority (let's call him Charlie), then Charlie can be provided constraints (in the form of instructions) to 'ask' the appropriate questions in a some order specified by the application (for example), for example phase flip first, followed by bit flip, OR bit flip first, followed by phase flip. (The principle is continued to any number of permutations, as needed for the 'application fragments' we wish to synthesize in the FPGA substructure.

Untangling the Ordering Spaghetti

One way to look at the event ordering problem is to consider using the above mechanism to 'pause the pipeline' in the FPGA. The design challenge is to do this in a way that does not introduce deadlocks or single points of failure (where a paused pipeline subsequently fails and goes zombie/AWOL).

One solution is to notice that, with two tokens, we can keep one token circulating (as a liveness mechanism), while the cell executes some application fragment (function) on behalf of the other. For example, If Alice has been asked to 'do something' with Bob's token, she can keep her simple liveness token circulating while she performs the execution requested by Bob.

Another solution is to consider a mechanism where liveness continues with a black Petri token, while the execution proceeds with the colored (serializability token).

Locking (holding a token to pause the pipeline) is therefore only possible with a colored Petri token (not a black Petri Token, which is for liveness only, and can never be changed by the node that doesn't own it).

TCP/IP is a forward-in-time-only protocol.

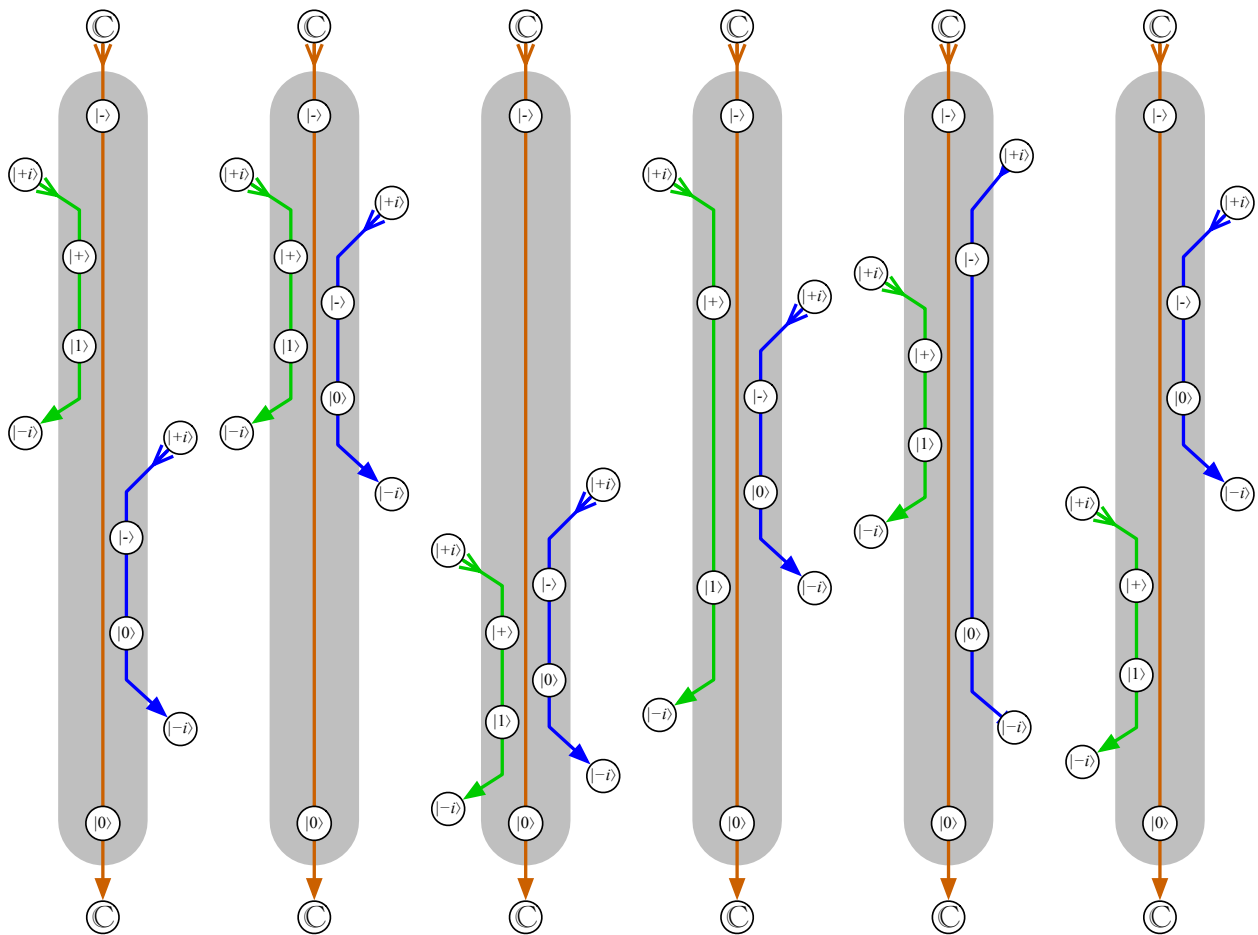


Figure 6: Charlie Serializability

Product Positioning

We have two things going on where we add value:

Loose Coupling (Scales) Through Stacked Tree Routing rooted on each node/smartNIC.

Tightly Coupled Dense configuration and liveness. Doesn't scale, but we don't need (or want) it to.

The former is the ultimate 'loose coupling' for processes in the FPGA, or in a processor core on the FPGA, can fire and forget down the tree branches without concern for how far away the destination node (or leaf) may be.

We can put NonStop on top of this and get rid of the cluster management. (Makes their life much easier).

Use Cases

The high speed trading industry need ultimate low-latency focussed on mechanisms to do algorithms in the FPGA's. They are not concerned about failures, if they see a failure, they drop the operation and move on to the next one. There is no time to waste. Bringing a feed into the FPGA, creating a response within the FPGA. The goal is to minimize latency from tick to trade. It is far too slow to go over the PCIe bus to the host processor to make decisions, so decisions are made in the Logic of the FPGAs.

Common Cloud Applications, such as Google's search, would like to minimize end to end latency from applications running in the host processors.

We anticipate a closer linking of functions in the processors and operations on the network.

The hierarchy of needs includes

1. Lowest Possible Latency – ‘register’ operations done in the FPGA, one or two slice turnaround (depending on SerDes constraints). We conceive of data structures as a set of registers. Eight 8-byte (64-bit) registers fit in one 64-byte packet.
2. SRAM latency, – cost 6 clocks but larger data structures are available for the applications to use.
3. Processor Core Latency – integrating with an ARM or RISC-V core with special instructions.

Temporal Cluster Intimacy

Design Choices

Do we adopt variable slice frames, or is the Flit size simply 8 bytes (or 16 bytes because that’s the minimum amount of information that can be sent out on the transmit channel of the SerDes)?

Do we adopt 64-byte ‘nominal’ size frames, to match Ethernet? If we do have our Liveness Tensor information sent out in a nominal 32, 48 or 64-byte packet, it would be helpful to have an pre-empt signal (where the receiver responds to the transmitted slice with a ‘jam’ signal, which says don’t bother sending the rest of this frame, I have something more important to do). Higher priority packets can pre-empt lower priority status information. The [Slice-Engine.pdf](#) doesn’t mind discarding status information, but it will make sure that multi-phase ‘transactions’ are completed, or at least terminated cleanly.

If we want the ultimate low latency, there may be opportunities to reduce the packet overheads. Preamble, and Inter-Record Gap (IRG).

Another major shift is to recognize that the synchronization boundary of the two SmartNIC’s connected back to back could be tied together for all transaction type information. The Cluster synchronization manager (Carol or Charlie) can observe or manipulate the synchronization

Many systems in the Telco industry are plesiochronous: they are close, but not perfectly synchronized. The SerDes can work the same way, slipping a cycle or two to get back into sync if the phase is not within design limits. Our system can go one step further: assuming that algorithms implemented by state machines (see [State-Machine.pdf](#)). An enhanced future product could tie the transmit/receive channels together so they are locked in sync, or even lock the entire loop transmit-recieve-transmit-recieve for Alice and Bob, so that they behave as a single synchronous clock domain.

This obviously doesn’t scale, because single synchronous clock domains are always going to be limited. But having an “adaptive” architecture that spans only the FPGA substructure one tile at a time, or across two links (from one link to another within the same cell) that need to forward packets is more workable. We leave this approach for exploration in a future design.